Flagpost

USER GUIDE

Admin Guide

Operating a Flagpost site

For **Site administrators** · Flagpost **1.7.1-src** · 18 October 2026

Contents

1	Becoming the administrator	3
2	Site settings & branding	5
3	Users & roles	7
4	Sign-in options	10
5	Competitions	12
6	Infrastructure: instances & the code judge	14
7	Pages, oversight & backups	17
8	FAQ	21

CHAPTER 1

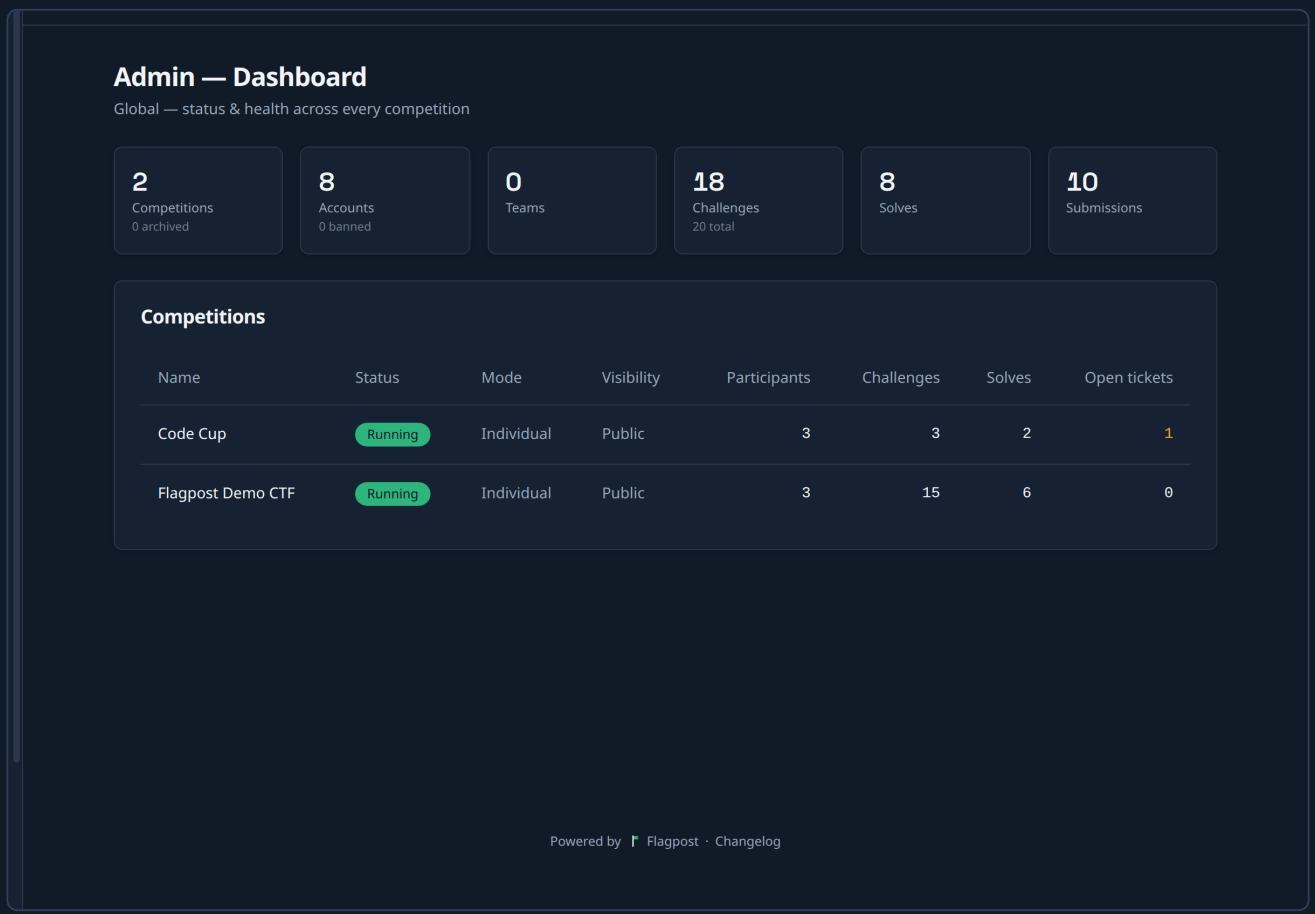
Becoming the administrator

A fresh Flagpost install ships with **no administrator account** — there is no default password to change, because there is no default login to leak. The first visit is greeted by the **setup wizard**: **Account** (the owner's username, optional email and password), **Branding** (platform name, palette and accent — the logo comes later) and **Options** (public registration open or closed; daily update checks on or off). Until it completes, public registration stays closed and the site reports itself as unconfigured.

> **SETUP RUNS ONCE**
>
> The wizard never reopens on a configured install — not even if the site is somehow left with no administrator — because an open wizard on a live site would be an anonymous takeover. Flagpost instead refuses to ban, delete or demote the last active Administrator. Keep two.

Your administrator account is global: it carries the site-wide permissions this guide covers — users, roles, sign-in options, site settings, the judge and instance infrastructure, and every competition on the instance. Day-to-day event work (authoring challenges, running a competition) is the Judge guide; many admins wear both hats, but the surfaces are separate. Both guides are in the app under **User guides** in the top bar.

Your home: the Admin area



Admin → Dashboard: the site at a glance — competitions, accounts, teams, challenges, solves and submissions, and a health table per competition.

The **Admin** section of the sidebar gathers the global surfaces, each with a chapter or section below: Dashboard, Skills, Competitions, Users, Roles, Event log, Automations, Pages, Marketplace, and Site settings.

> **WHO SEES THE ADMIN SECTION**
>
> Anyone holding a global grant that unlocks one of its pages — and they see only the pages they can open. A colleague delegated just the event log gets an Admin section with just the Event log in it.

Staying current

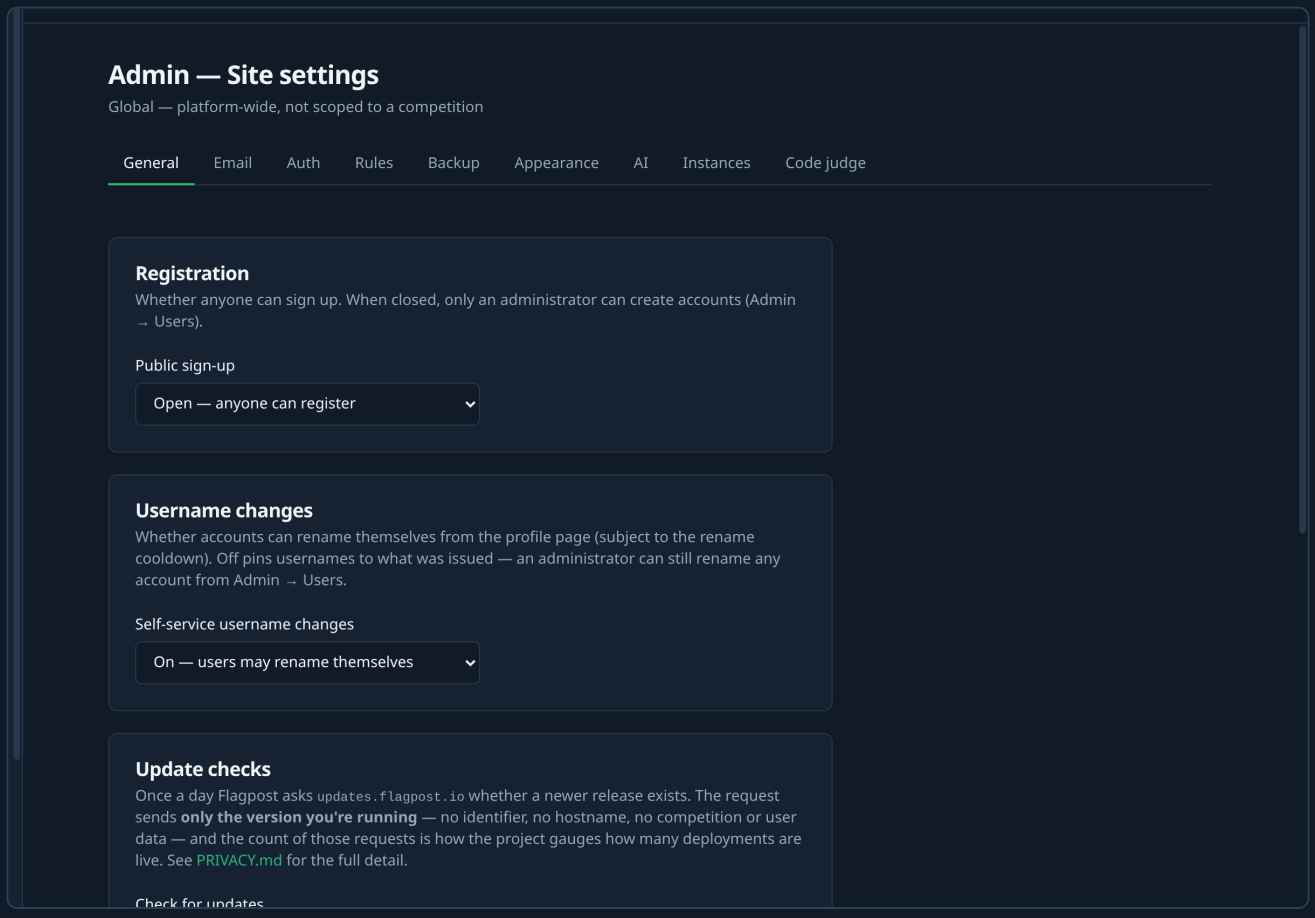
When a newer release exists, a dismissible banner tells administrators Flagpost X is available — you're running Y, with **How to update**; the Update checks card in Site settings shows the running version and the last check, and the footer's **Changelog** lists what each release shipped.

> **DELEGATE WITH ROLES, NOT SHARED LOGINS**
>
> Resist the shared admin account. Chapter 3 shows how to grant a colleague exactly the site-wide permissions they need under their own identity — the event log is only as useful as the names in it.

CHAPTER 2

Site settings & branding

Admin → Site settings holds the platform-wide configuration, in tabs: General, Email, Auth, Rules, Backup, Appearance, AI, Instances and Code judge. Auth, AI, Instances and Code judge each need their own grant to appear (chapter 3).



Global — platform-wide, not scoped to a competition.

General

- **Registration** — public sign-up open, or closed (then only administrators create accounts, or a closed-policy identity provider admits them — chapter 4).
- **Username changes** — whether users may rename themselves (once every 30 days). Switch it off to pin usernames to what was issued; administrators can still rename anyone from Admin → Users.
- **Update checks** — once a day the site asks whether a newer release exists. The request carries **only the running version** — no identifier, hostname or user data. Turn it off if policy demands.
- **Data retention** — when on, an archived competition is permanently purged after the retention period. The archive dialog shows the exact date; unarchiving cancels the clock.

Email

SMTP for outbound mail — password-reset and email-verification links, the "your email was changed" notice, and the automation send email action (notifications themselves are in-app only). Without SMTP those flows quietly do nothing — a password reset still says a link is on its way — so configure it early. The password field is write-only.

Two registration controls live here too: an **email domain allowlist** (subdomains included; makes email mandatory at sign-up; a rejected sign-up sees only a generic error; also applies to open-policy providers), and **email verification** — a self-registered account must confirm its address before it can join a competition. It needs SMTP, and admin-created accounts are exempt.

Appearance

Site-wide theming — there is deliberately no per-competition theming. The **platform name**, a **palette** (five built-in, or a custom theme) and an **accent**; your **logo** (PNG, SVG, WebP or GIF, up to 1 MB); an animated **sign-in background** on dark palettes; and a public **sign-in notice**. **Custom themes** are full colour packs built from your brand colours — **New theme** or **Upload theme**, with a live preview — selected in the palette grid like any other. Users may override the palette for themselves from the top bar's Theme menu. The "Powered by Flagpost" footer stays regardless of branding.

Rules

The site-wide **Rules / code of conduct** shown before joining any competition that hasn't set its own override — edit it and every such competition changes at once. **Display only** shows the rules without requiring acceptance; empty means no rules and no prompt.

AI

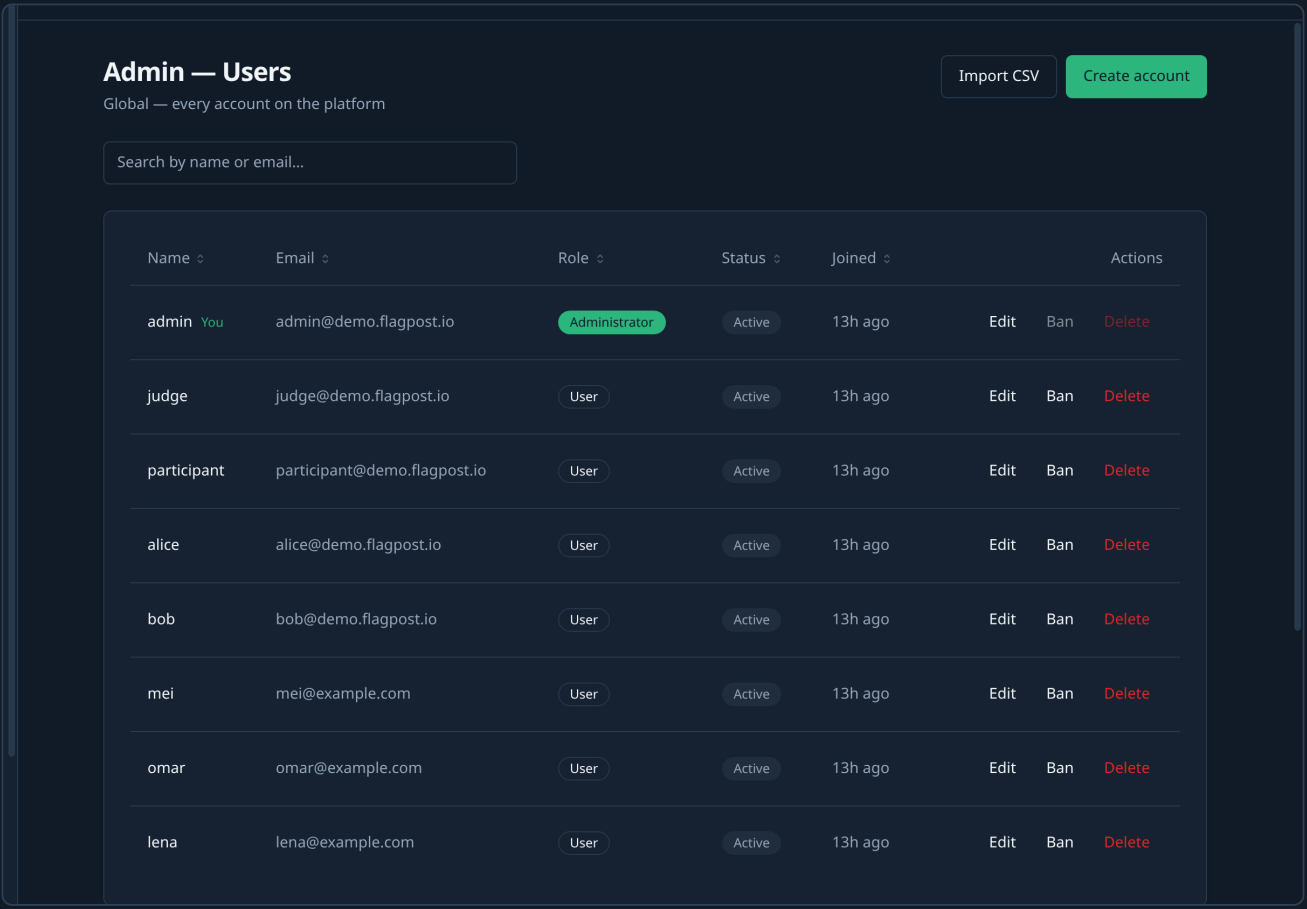
The optional assistants' provider: any OpenAI-compatible endpoint (**Base URL**, a **Model** that supports tool calling, an optional **API key**), **Response limits**, and **Test connection**. It ships **off** and can't be switched on until a URL and model are saved; nothing calls out until then, and chats flow to the provider you chose — read the privacy notes before enabling it on a site with minors or sensitive data. Whether competitors get an assistant is each competition's choice (Settings → Assistant, Judge guide).

Backup, Auth, Instances and Code judge have chapters of their own.

CHAPTER 3

Users & roles

The users directory



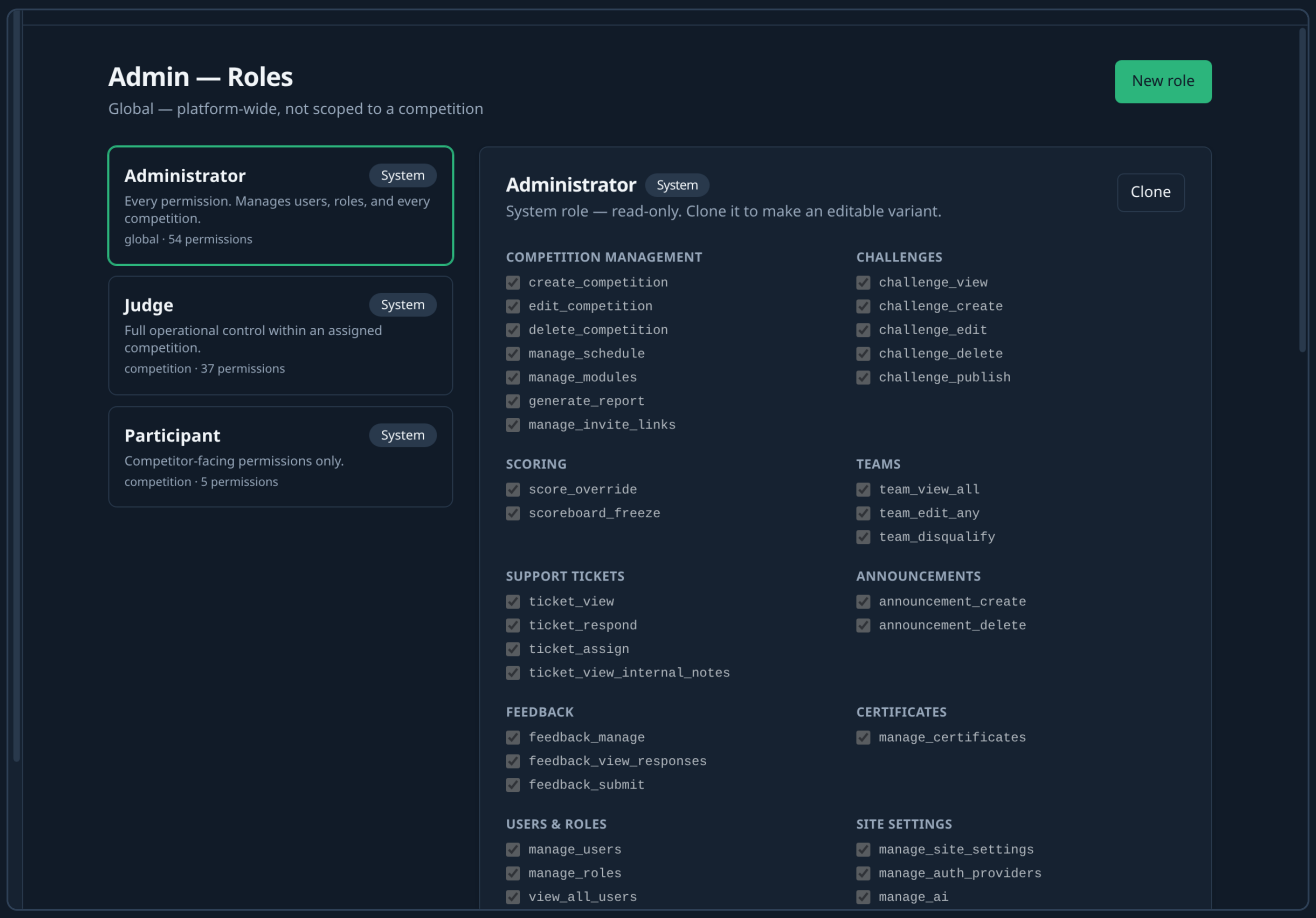
Every account on the site — search, create, import, moderate — and every API token below.

Admin → Users lists every account. **Create account** adds one with a password that works immediately; **Import CSV** bulk-creates a roster (up to 200 rows, with an optional role per row — the file carries plaintext passwords, so treat it as sensitive and delete it afterwards). Identity is username-first: the display name is the unique login handle, and email is optional unless your registration settings require it.

Per account: **Edit** (rename, change email, or set a new password — which signs the user out everywhere), **Ban** / **Unban** (a ban signs them out at once and blocks sign-in; everything they did survives with attribution), **Remove picture**, and **Delete** (permanent — submissions, tickets and team memberships included; prefer a ban). You cannot ban or delete yourself, or the last active Administrator.

The **API tokens** panel at the bottom lists every personal token on the platform; **Revoke** kills any of them immediately. That's your lever when a credential leaks.

Roles: permissions as data



Each role is a named bundle of granular permissions, split between global and competition scope. Roles are granted in the Assignments card further down the page.

Admin → **Roles** manages the catalogue. Three system roles ship — **Administrator** (global), **Judge** and **Participant** (competition- scoped) — read-only, and they track the platform: a new permission in an update reaches them automatically. **Clone** one for an editable variant, or **New role** with a name and a **Scope**:

- **Global** permissions (manage users, roles, site settings, create competitions, the event log, marketplace, judge and instance infrastructure…) apply site-wide.
- **Competition** permissions (edit challenges, manage the schedule, tickets, announcements, judging, instances…) apply inside whichever competition the role is granted for.

So “ticket handler who can also see analytics” or “site moderator who manages users but never touches events” are each a five-minute custom role, not a code change.

Granting a role

The **Assignments** card: pick a **User**, a **Role**, and a **Competition** (or **Site-wide** for a global role), then **Assign**. You can’t grant, or take away, more than you hold yourself, and the last Administrator can’t be unassigned. The same goes for accounts: someone delegated user management looks after ordinary players, but can’t reset, ban or delete a Judge or an Administrator. Someone who signs in through an identity provider arrives with no role; they become a Participant per competition as they join, and staff roles are yours to grant — sign-in method never implies privilege.

GRANT THE LEAST THAT WORKS

Start people on the narrowest role that covers their job and widen on request. The Auth, AI, Instances and Code judge tabs each have their own grant, as do most Admin pages, so an Auth delegate needn't be a full administrator; General, Email, Rules, Backup and Appearance share manage site settings — though changing the mail server or the From address needs an Administrator, since password-reset links travel through them. Treat manage auth providers as administrator-level too.

CHAPTER 4

Sign-in options

Local username/password sign-in works out of the box. **Site settings** → **Auth** (its own grant) adds external identity providers alongside it — each redirect-based one appears as a **Sign in with …** button.

What you can connect

KIND	TYPICAL USE
Quick set up presets	Google, Microsoft, Microsoft (multi-tenant), GitHub, Discord — paste client credentials and go
OpenID Connect	any standards-compliant IdP (Okta, Keycloak, Auth0, Entra…)
OAuth 2.0 (no OpenID Connect)	providers without an ID token — identity comes from a userinfo call and a claim map (what GitHub and Discord use)
SAML 2.0	campus and enterprise IdPs that predate OIDC
LDAP / Active Directory	direct directory sign-in — no button; the normal login form checks the directory when local verification fails

New providers **start disabled**: configure, register the **Redirect URI** (or the **SP metadata** URL for SAML) at the IdP, test, then **Enable**. The protocol and slug are fixed after creation. Prefer **Disable** to **Delete** — a user who only ever signed in through a provider has no password to fall back on.

> **MULTI-TENANT MICROSOFT TRUSTS EVERY TENANT**
>
> The multi-tenant preset admits any Microsoft work or school account. Pair it with the email domain allowlist (Site settings → Email) to keep the roster to your organisation.

Sign-in policy

An OpenID Connect or OAuth 2.0 provider is **Open** or **Closed**; SAML and LDAP are always closed.

- **Open — public provider, registration rules apply.** A new account must pass the same checks as the sign-up form: registration open, and the email domain allowlist. Right for Google or GitHub on a public event.
- **Closed — being in this directory is the admission.** Anyone who can sign in at that IdP gets in, even while public registration is closed. Right for your own corporate or campus directory.

One account model

A first sign-in creates an account with **no role** — it becomes a Participant per competition on joining, like self-registration — and later sign-ins link by the provider's stable subject, so a changed email at the IdP doesn't fork the account. A first sign-in attaches to an existing local account with the same email only when the address is trusted (asserted verified by an open provider, or **Trust this provider's**

email addresses ticked for a closed one). If the local account's email was never verified, the sign-in is refused rather than linked; if the provider's address isn't trusted, no linking is attempted and a separate account is created with no email on it.

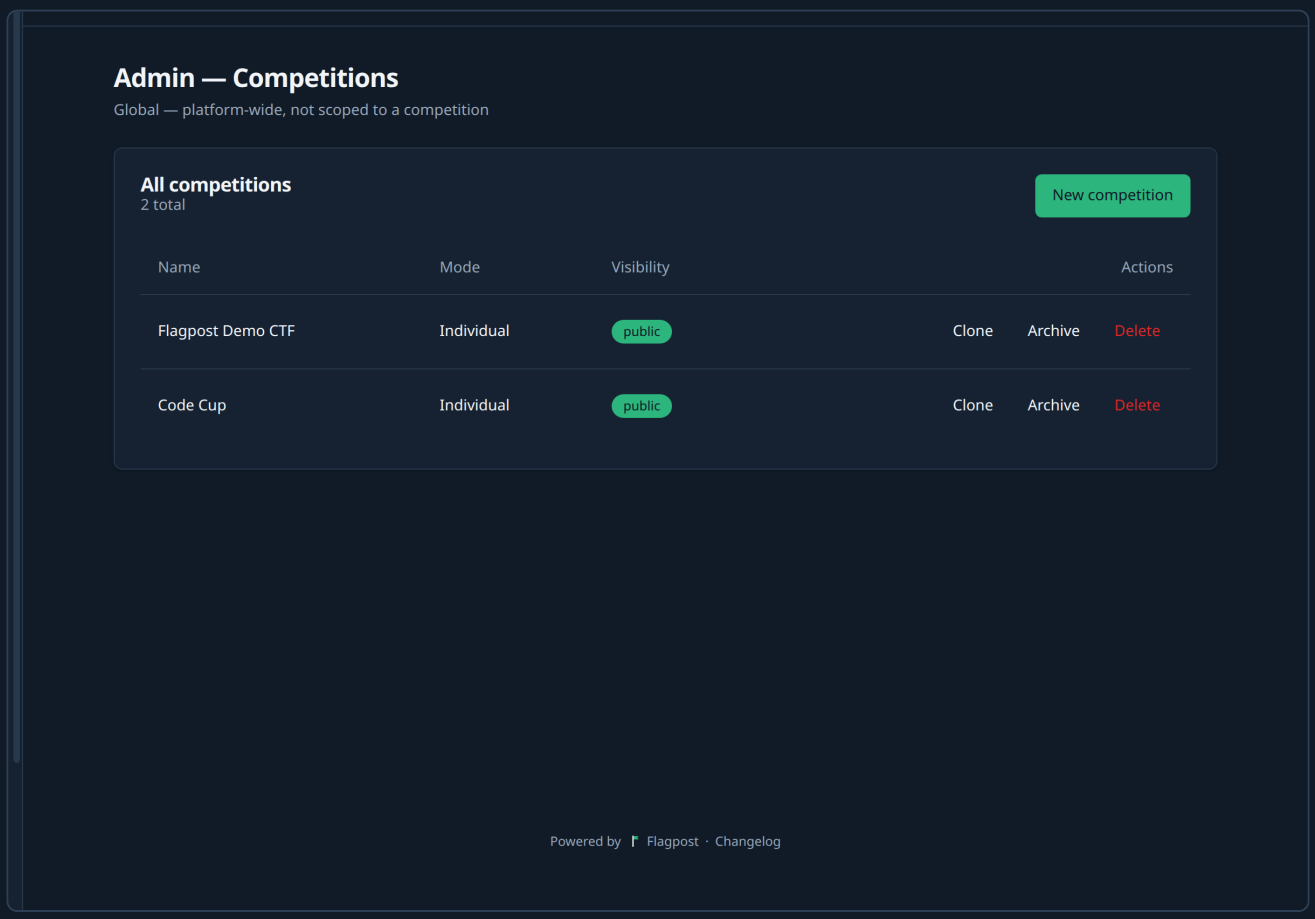
Practical notes

- **Local login stays available** as the break-glass path. A provider- created user has no usable local password until one is set — through **Forgot password?** (needs an email and SMTP) or by you in Admin → Users → Edit — and they'll need one to create API tokens. Keep at least one local administrator account you control.
- Provider secrets are stored encrypted and never shown back; they're left out of backups.
- A sign-in must finish in the browser that started it; a callback opened elsewhere gets That sign-in link has expired or was already used.
- Redirect-based providers need the IdP to know your site's public URL — if a provider rejects the callback, the URL registered at the IdP and the one the site runs behind disagree. The deployment side lives at docs.flagpost.io.

CHAPTER 5

Competitions

Competitions are the tenancy unit: each has its own challenges, roster, teams, scoreboard, settings and staff. **Admin → Competitions** manages them site-wide; **Admin → Dashboard** shows each one's status and health.



Every competition on the instance, with its mode, visibility and lifecycle actions.

Creating

New competition asks for a name and description, the **Participation mode** (team or individual), **Visibility** (Private by default, or Public), whether to publish a **public scoreboard**, an optional **Starts** / **Ends** schedule, and which optional **Modules** the event runs — AI Assistants, Analytics, Automations, Certificates, Challenge Instances, Code Judge, Feedback & Surveys, Post-Event Reports. All start on; switch off what the event won't use (judges can change this later in the competition's Settings → Modules). Rules, divisions, registration fields and the registration window are configured on the competition afterwards, usually by its judges.

> **THREE MODULES NEED SITE SETUP FIRST**
>
> AI Assistants, Challenge Instances and Code Judge do nothing until you've configured them under Site settings (AI — chapter 2; Instances and Code judge — chapter 6). The per-competition toggle alone isn't enough.

Access

A **public** competition is listed in the lobby and the competition switcher and joinable by anyone signed in. A **private** one is entered only with its invite code or a **join link** the judges mint (Settings → Sharing). Public registration being open or closed (chapter 2) composes with this: a closed site with a private competition is a fully provisioned, private event. If email verification is on, it gates joining too.

Cloning

Clone copies a competition's settings, categories, challenges (with their flags), hints, files, surveys and module toggles into a fresh event — without participants, scores, tickets or a schedule. It's the fast path for recurring events, and the safe place for staff to rehearse flag submissions (their solves on the live event are real).

Archiving and deletion

- **Archive** closes a finished event and hides it from the switcher and lobby; everything is retained and it can be unarchived at any time — unless **data retention** (chapter 2) is on, in which case the archive dialog shows the exact date it will be purged.
- **Delete** is immediate and permanent — challenges, teams, submissions, scores, tickets, surveys. Prefer archive unless you genuinely need the data gone; if you might ever want the history, export a backup first (chapter 7).

CHAPTER 6

Infrastructure: instances & the code judge

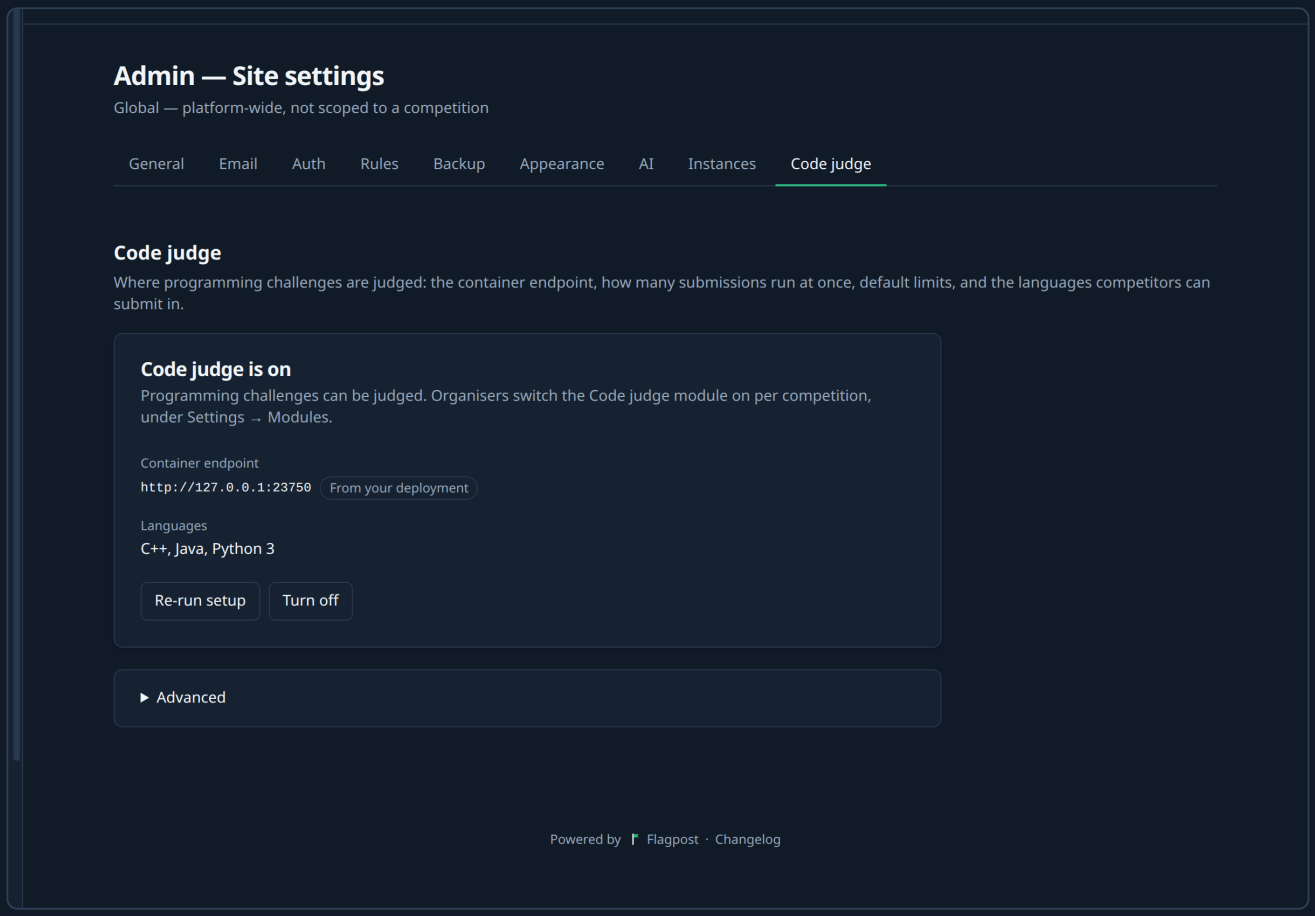
Two optional modules run competitor-controlled code and need a container runtime behind them. Both ship **off**, each has its own grant, and neither does anything — however many competitions switch the module on — until you set it up here. Where they use Docker (the code judge always; challenge instances on the Docker backend) they talk to it through a **least-privilege socket proxy**, never the raw socket; the Kubernetes backend uses a namespace-scoped service account instead. Standing the proxy up is operator work (docs.flagpost.io), and the judge's setup shows you the one-line command.

Site settings → Instances

Challenge instances hand each team its own running copy of a service (Judge guide, chapter 4). The **Provider** card holds **Status**, the **Backend** (Docker or Kubernetes), the proxy's **Endpoint URL**, the **Public host** competitors connect to, an optional **Registry credential** for private images, an **HTTP base domain** for web challenges (needs wildcard DNS and a wildcard certificate), and a per-participant **Spawn rate limit**. **Exposure & networking** sets the host **port range** to open in your firewall and the **Egress policy** (Deny by default); **Default resource limits** sets CPUs, memory, PIDs and a **global concurrent instances** ceiling. Kubernetes adds a namespace-scoped service-account token, CA certificate, ingress class, pull secret and cluster CIDRs.

Save changes, then **Test connection**: a staged check that reports each step OK or Failed — reachability, privilege posture, network isolation, an image pull and a probe run. Nothing can be enabled until an endpoint and public host are saved (for Kubernetes, a token). Deployments written for Docker run unchanged on Kubernetes.

Site settings → Code judge



Where programming challenges are judged: the connection, the languages competitors can use, and the default limits problems inherit.

The code judge runs competitors' programs, each submission in a short-lived container with no network. Setup is a guided three-step flow:

1. **Connect** — the **Proxy address** (or Detected from your deployment if the compose overlay set it) and **Check connection**, which must pass all four: Proxy reachable, Proxy locked down, Judge harness installed, Test judging ran. A proxy that isn't locked down would hand submissions a path to your server's Docker, so setup won't continue until it is.
2. **Languages** — tick what competitors may submit in. Nine are built in (C, C++, Rust, Go, Java, C#, JavaScript, Python 3, PyPy 3), each on a pinned official image downloaded on first use.
3. **Review and enable.**

Once on, the tab shows a summary with **Re-run setup** and **Turn off**. **Advanced** exposes the **Executor** (**Judgings at once** — one is the lightest and the fairest for time limits on shared hardware), the **Default limits** every problem inherits, and the **Languages** list: each has a **Time multiplier** so slower languages aren't penalised by limits tuned for C — C/C++/Rust 1.0, Go 1.5, Java/C#/JavaScript/PyPy 2.0, Python 3.0 — plus **Add custom language** for anything else. For a strict ICPC-style event, set every multiplier to 1.0.

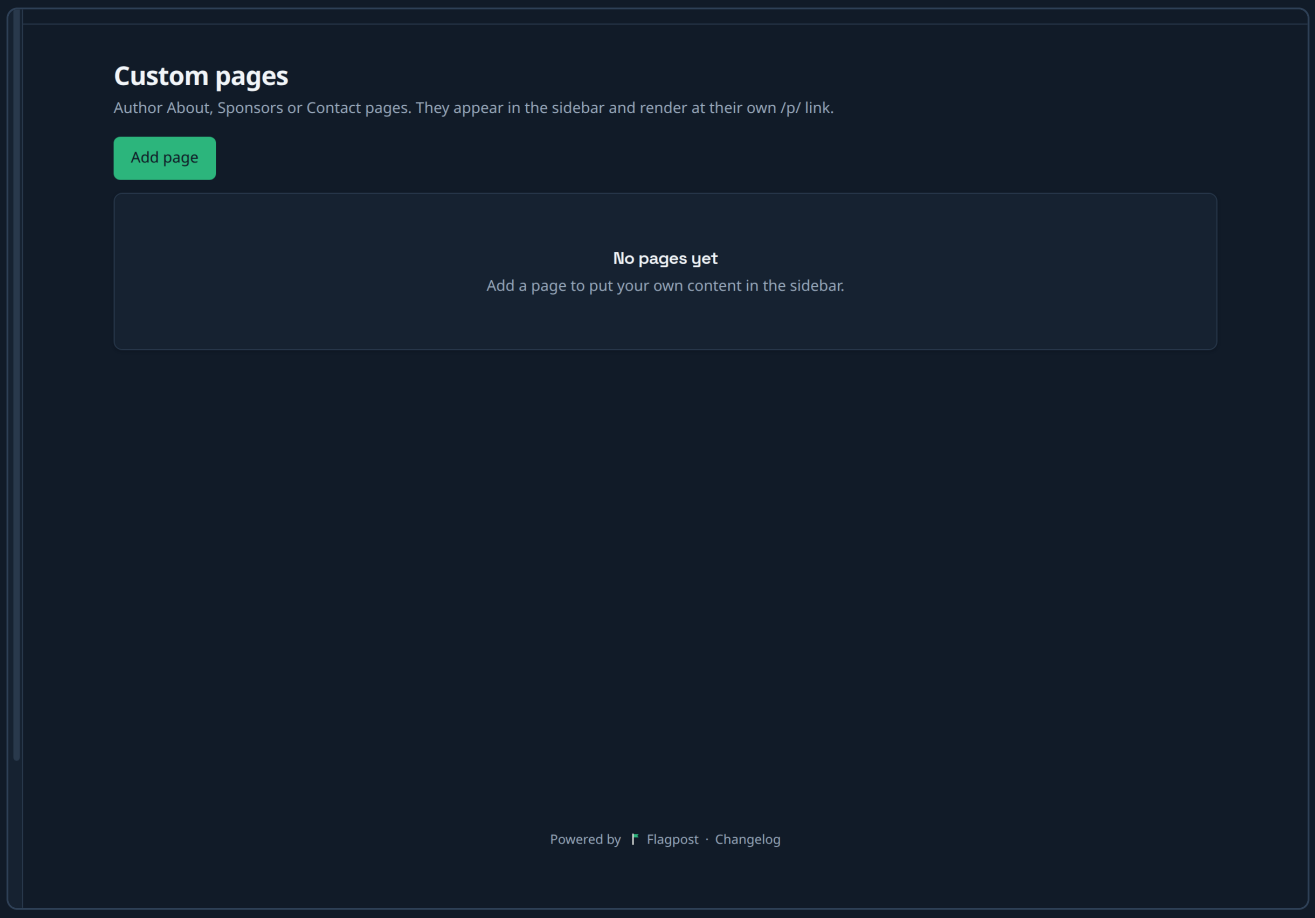
> **NOT IN BACKUPS**
>
> The judge and instance configuration — endpoints, credentials, languages — is deliberately excluded from the platform backup: it names host-specific infrastructure and runs code. After restoring onto a new host, re-run both setups.

Organisers then switch the **Code Judge** (or **Challenge Instances**) module on per competition — it's on by default — and author away.

CHAPTER 7

Pages, oversight & backups

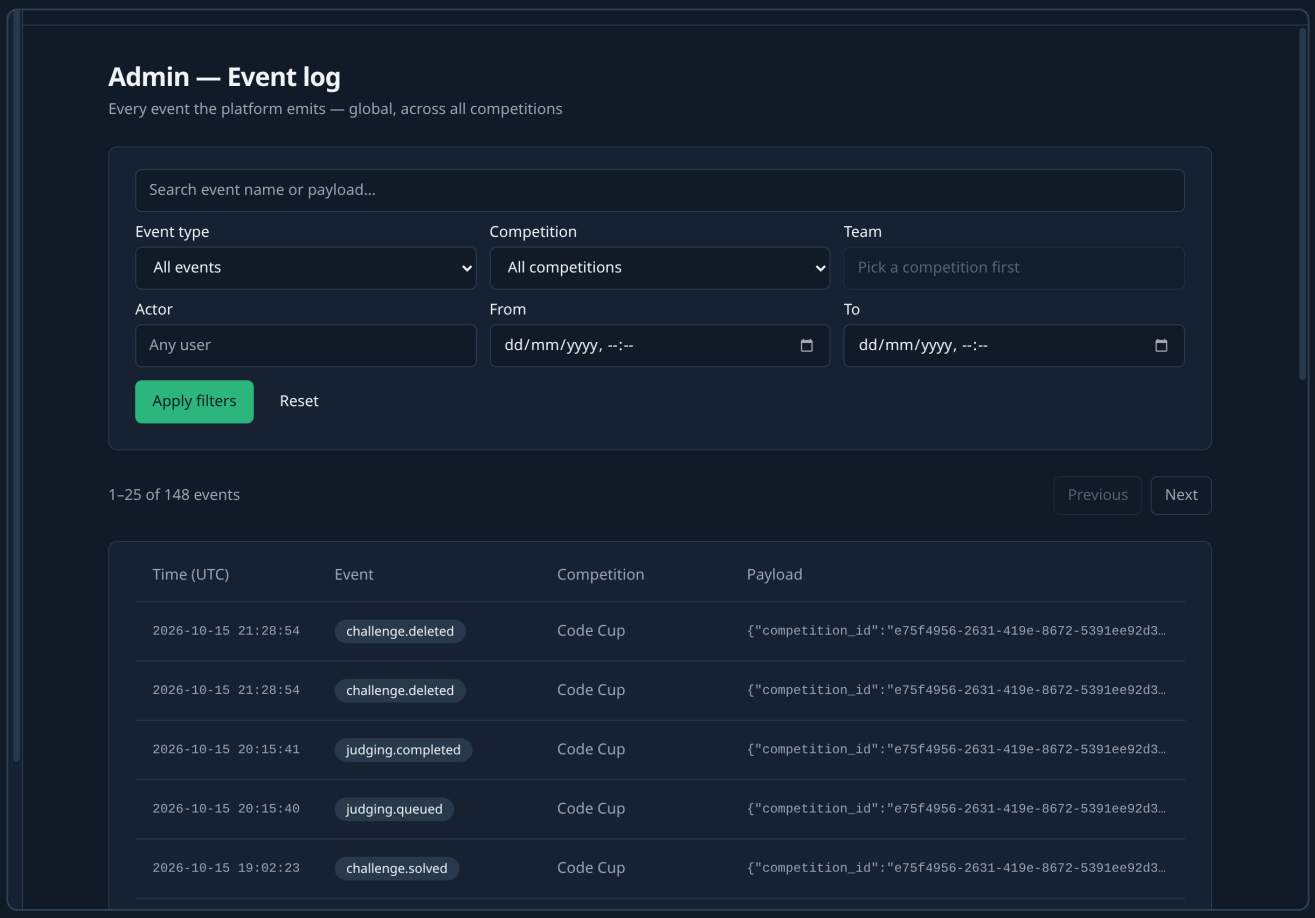
Custom pages



Admin → Pages: rich-text pages with a sidebar icon, a URL slug and an audience.

Pages are **site-wide** — full rules, prizes, venue or sponsor information — each with an icon, a slug (`/p/slug`; changing it breaks links), and an audience: **Signed-in users only** or **Anyone, including signed-out visitors** (public pages are also linked from the sign-in screen). Drafts stay invisible until published; **Move up** / **Move down** orders the sidebar.

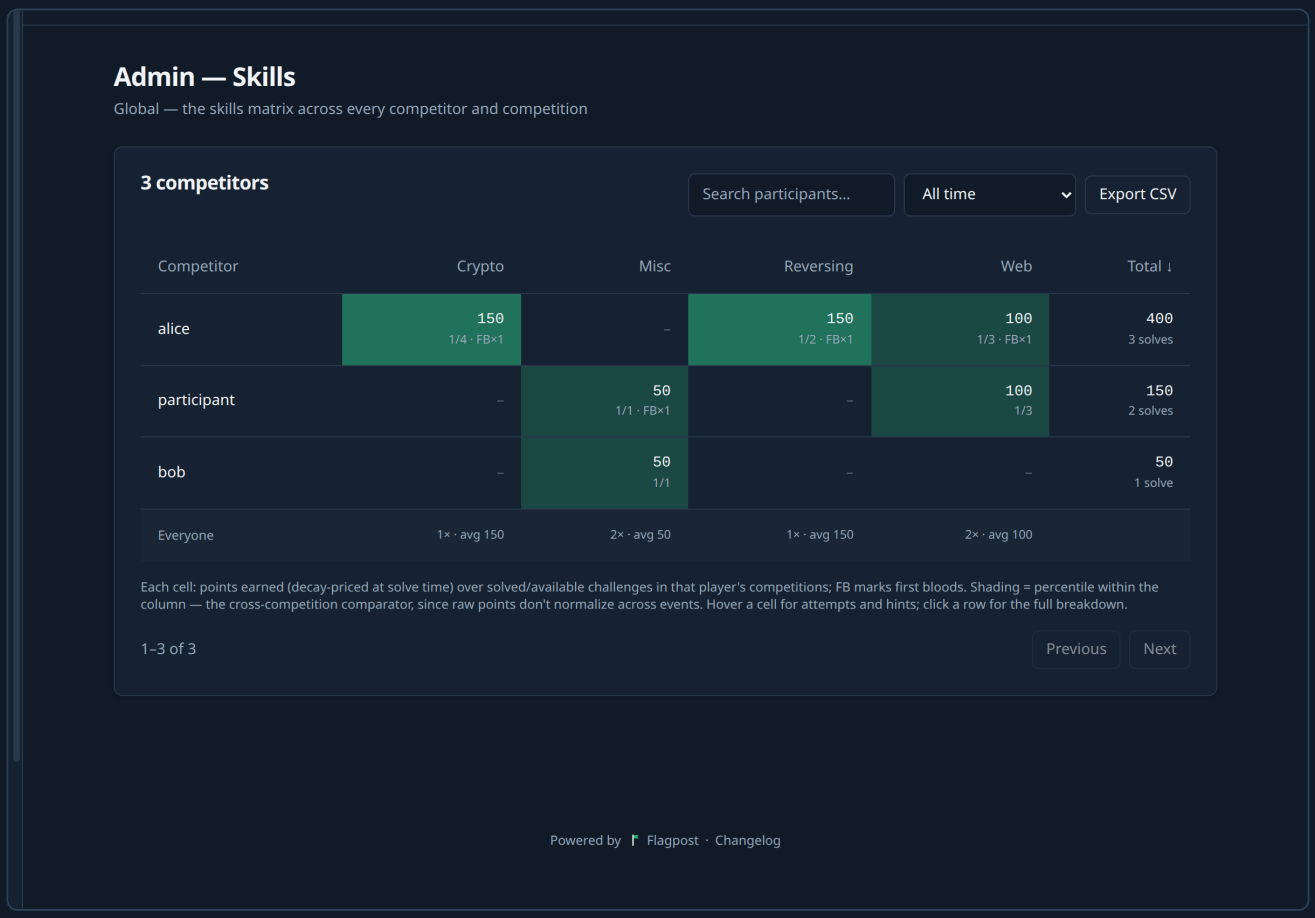
The event log



Admin → Event log: every event the platform emits, across every competition.

Everything that happens lands here — staff actions (publishes, role changes, bans, score adjustments, overridden verdicts), security-relevant events, and gameplay (solves and attempts, flag-sharing detections). Filter by text, **Event type**, **Competition**, **Team**, **Actor** and a time range (UTC); click a row for its full payload. It's the first stop when something looks off mid-event, and the record that makes post-event disputes short. Judges don't have it — point them at their dashboard's Moderation feed.

Skills



Admin → Skills: every competitor's strength per category, across every competition.

One row per competitor, one column per skill (a challenge category), accumulated across every event they've played. Each cell shows points over solved/available challenges and first bloods, shaded as a percentile within the column — the right comparator, since raw points don't normalise across events. Search, narrow the **Activity window**, sort, click a row for the full breakdown, or **Export CSV**. Competitors see only their own skills (Profile → Skills).

Marketplace

Admin → Marketplace installs **content packs** from a registry: paste an **Import code**, **Resolve**, review it (Signed or Unsigned, the publisher, what it requests), pick a **Target competition** for a challenge pack, and **Install** — signatures are verified locally first. Challenge packs land as challenges; brand-theme packs appear under Appearance. **Registry & trust** holds the on/off switch (off blocks every install path), the **Registry URL** — point it at a private mirror for air-gapped installs — the **Trust policy** and **Trusted keys**. The registry is contacted only when you resolve or install. Importing and configuring are separate grants.

Global automations

Admin → Automations holds **global rules** that run across every competition (per-competition rules are the judges'). Each shows what it fires on and how often; Edit, Enable or Disable, Delete. Webhook egress is locked down by design: no calls into private networks.

Backups

Site settings → **Backup** exports selected sections — site settings & branding, user accounts, roles & assignments, competitions (challenges, teams, scores, tickets…), automation rules, the audit log — to one JSON file, and imports one **additively**: records that already exist (a competition by name, a user by name or email) are skipped, never overwritten.

> **WHAT TRAVELS, AND WHAT DOESN'T**
>
> The file holds password and flag hashes, emails, uploaded files and programming test data — store it like a credential. Deliberately **left out**: identity-provider configuration, the SMTP password, AI settings and key, API tokens, sessions, and the judge and instance infrastructure. Re-enter those after a restore — a good moment to rotate them.

Take one before risky changes (imports, big deletions, upgrades) and after each event — restore-tested backups are the difference between an incident and an anecdote.

That's the site side. Event operations live in the **Judge guide**, competitor-facing behaviour in the **Competitor guide**, and deployment, scaling and integrations at **docs.flagpost.io**.

CHAPTER 8

FAQ

I've lost administrator access — how do I recover?

Another **Administrator** signs in and restores you — the reason to always have two — or use **Forgot password?** if your account has an email and SMTP is configured. The setup wizard does **not** reopen on a configured install, by design; beyond those two paths, recovery is an operator job at the deployment level (docs.flagpost.io).

Does closing registration affect existing users?

No — it only stops new public sign-ups. Existing accounts sign in as normal, **closed**-policy identity providers keep admitting their users, **open** ones stop creating accounts, and you can still create or import accounts from Admin → Users.

What does the platform send outside my network?

Only traffic you configure: SMTP mail, automation webhooks, identity-provider exchanges, the AI provider if enabled, the container runtime behind instances or the code judge (which pulls language images on first use), and the marketplace registry — contacted only when you resolve or install a pack. Plus one **daily update check that sends only the running version number**, which you can turn off in Site settings → General. There is no other phone-home.

What happens the first time someone signs in through SSO?

The account is created with **no role** and becomes a Participant per competition as they join. It's linked to the provider's stable subject, so a later email change at the IdP doesn't fork it; it only attaches to an existing local account when both sides' emails are trusted. Whether a new account is allowed while registration is closed depends on the provider's **Sign-in policy** (chapter 4). Staff roles are yours to grant afterwards.

Someone leaked an API token.

Admin → Users → **API tokens** lists every token on the platform with its holder; **Revoke** stops it authenticating immediately.

Can I move a backup to another instance?

Yes — upgrade the target first so the formats match, then import; it's **additive**, so existing records are never overwritten, and you can move only some sections. Identity providers, the SMTP password, AI settings, API tokens and the judge and instance configuration don't travel — re-enter them on the target (and rotate). The file carries password hashes and flags: move it like a credential.

How do I block an account without destroying its history?

Ban from Admin → Users: they're signed out immediately and blocked from signing in, while every solve, ticket and event entry survives with attribution intact. **Unban** restores access exactly as it was.

If I toggle a module off for a competition, what changes?

That competition loses the feature's surfaces while the toggle is off — nothing else on the site is affected. The reverse isn't symmetric for AI Assistants, Challenge Instances and Code Judge: switching those on per competition does nothing until the matching Site settings tab is configured and enabled.

A competitor says the scoreboard "lost" their programming solve.

Judges can rejudge or override verdicts, and the newest verdict counts — the **Event log** records who did what and why. Ask the competition's judges first; the override reason is visible to staff on the Judging page.